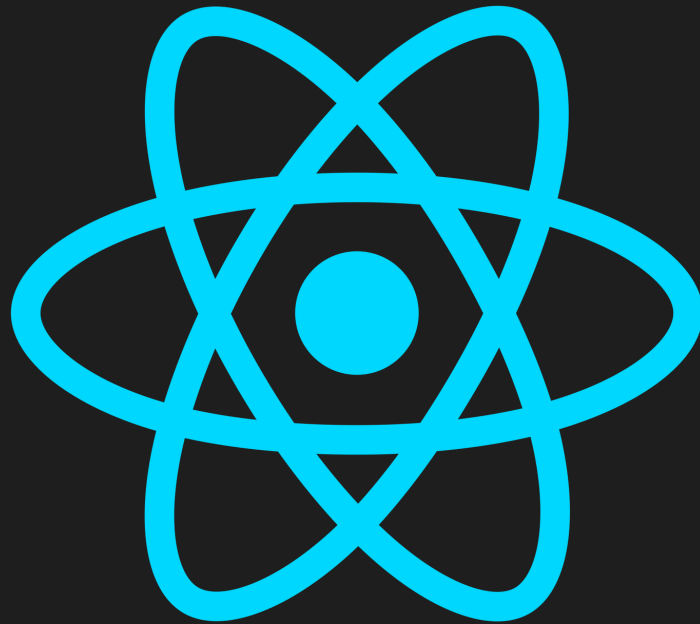


ملخص مختصر ل اهم

اساسيات React.JS



اعداد: Youssef Salah

محتوي الملخص

- يعني إيه React وليه نستخدمها؟
- إزاي تجهز بيئة العمل ل React
- فهم مبدأ ال Components في React
- ال Props في React إزاي تبعت بيانات بين ال Components
- ال State في React إزاي تخلي ال Component ديناميكي؟
- التعامل مع الأحداث (Events Handling) في React
- التعامل مع ال Conditional Rendering في React
- يعني ايه JSX وإزاي بيشتغل جوه React؟
- ال Lifecycle Methods في React؟
- التعامل مع ال Hooks في React
- التعامل مع ال Forms في React؟
- إدارة الحالة المتقدمة (Context API أو Redux)
- التعامل مع REST APIs في React
- التنقل بين الصفحات في (React Router)
- أهم أسئلة ما بعد الملخص ؟





يعني ايه React
وليه نستخدمها؟

يعني إيه React وليه نستخدمها؟

React.js هيا مكتبة JavaScript مفتوحة المصدر اتعملت عشان تسهل بناء واجهات المستخدم (UI) بطريقة مرنة وسريعة وبتساعدك إنك تكتب كود منظم وتشتغل بالمكونات وكمان تخلي موقعك أسرع بفضل مفهوم ال Virtual DOM

ليه React مش غيرها؟

1. السرعة

- بتستخدم ال Virtual DOM، فبتحدث الصفحة بشكل ذكي بدون ما تعمل إعادة تحميل كاملة.

2. إعادة الاستخدام

- المكونات في React تقدر تعيد استخدامها في أكثر من مكان في التطبيق.

3. مجتمع كبير

- لأنها مفتوحة المصدر، بتلاقي فيها دعم كبير وكثير من الأدوات الجاهزة

مناسبة لمين؟

- التطبيقات الديناميكية الي فيها بيانات بتتغير كثير

- التطبيقات الي فيها واجهة مستخدم معقدة

- Single Page Applications (SPA)

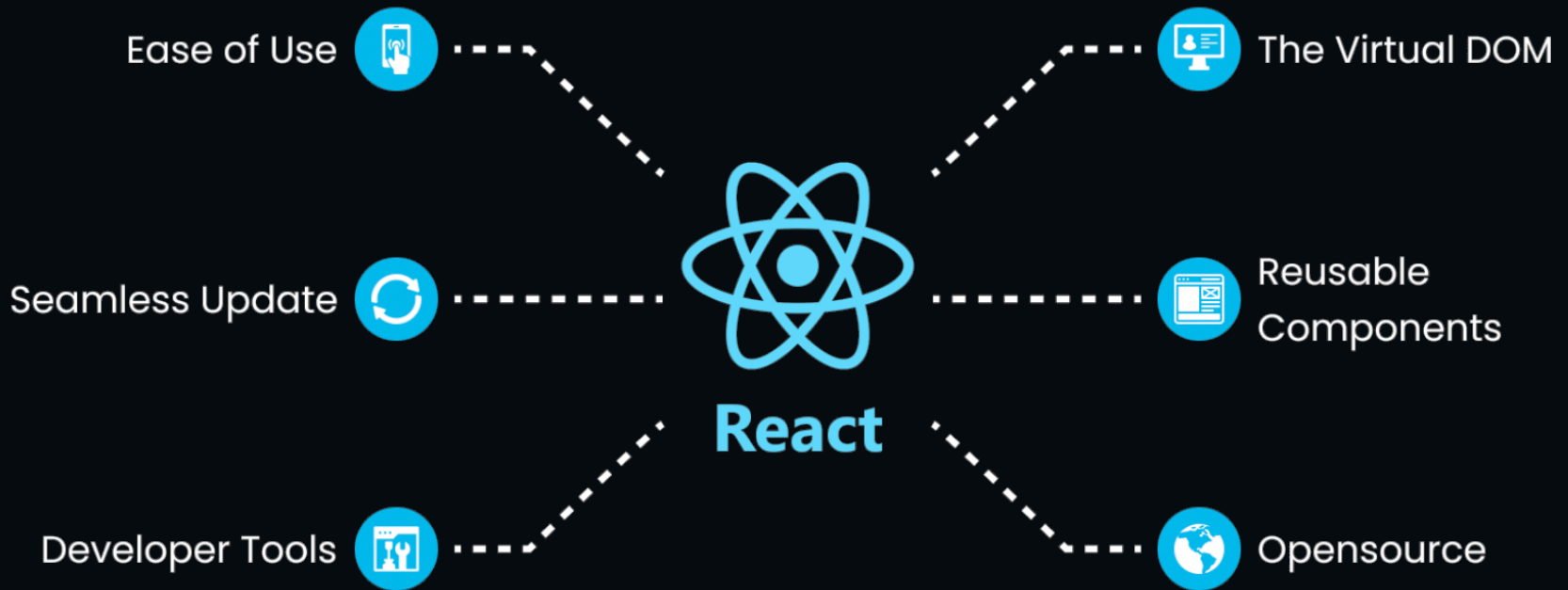
تطبيقات تستخدم React

- **Facebook** - لأن React أصلاً معمولة من فريق فيسبوك.
- **Netflix** - يستخدموها عشان الأداء العالي.
- **WhatsApp Web** - لأنها خفيفة وسريعة في التعامل مع الال.



ليه أتعلم React.js؟

- ال Automate tasks وبتقدر تعمل برامج صغيرة تسهل الشغل اليومي
- فهم أفضل ل ازاى الكمبيوتر بيشتغل ولوحدها بتعلمك كيف تتواصل مع نظام التشغيل.
- يعني لوحدها قوية لو عايز لو عايز تكتب برامج بسيطة لأتمتة مهامك اليومية





ازای تجهیز بیئة العمل لـ React

قبل ما نبدأ.. محتاجين نعرف إيه؟

عشان تشتغل على ال React لازم يكون عندك الحاجات دي:

- ✓ **Node.js & npm** عشان تشتغل React وتدير الحزم (Packages)
- ✓ **VS Code** أو أي محرر اكواد عشان تكتب الكود بطريقة مريحة
- ✓ **متصفح زي Chrome** عشان تشوف التعديلات بسرعة



وعشان Node.js و npm فلازم أول حاجة لو مش عندك ال Node.js روح نزلّه من الموقع الرسمي

بعد كده هتأكد إنه اتثبت صح وتكتب الأوامر دي في ال Terminal

```
node -v --- هتعرض إصدار ال Node.js
npm -v --- هتعرض إصدار ال npm
```

بعد كده هتظهر لك إصداراتهم وبكده التثبيت نجح



• هنجي بقي لتثبيت محرر اكواد مناسب ؟

أفضل محرر كود تستخدمه مع React هو **VS Code**

• نزل VS Code من هنا: <https://code.visualstudio.com>

• ده محرر خفيف وسريع وفيه إضافات كتير مفيدة لكتابة كود React



طب ازاي اعمل مشروع React ؟

طبعا عندك طريقتين عشان تعمل مشروع React بس هنستخدم الطريقة الأسهل

✓ باستخدام Create React App (الطريقة العادية)

```
npx create-react-app my-app  
cd my-app  
npm start
```

- **npx** بيشغل ال create-react-app من غير ما تحتاج تثبته
- **npm start** بيشغل السيرفر المحلي ويخلي الموقع يتحدث تلقائيًا مع كل تعديل

الطريقة الثانية باستخدام Vite

(عشان هوا الأسرع) لانك لو بتدور على أداء أعلى يبقى Vite ممتاز ليك

```
npm create vite@latest my-app --template react  
cd my-app  
npm install  
npm run dev
```

طبعا هنا ال Vite أسرع بكثير في انه يشغل المشاريع وتحميل الملفات

ايه هيا بنية مشروع ال React ؟

لما بتعمل مشروع React هتلاقي ملفات ومجلدات جاهزة أهمهم

→ /src  هنا بتحت كل الأكواد الخاصة بالمشروع

→ /public  فيه ملف index.html اللي بيتحميل التطبيق جواه

→ package.json  ملف بيحتوي على بيانات المشروع والمكتبات اللي بتستخدمها

• ازاي بقي تشغل المشروع؟

بعد ما تعمل كل ده شغل المشروع بالأمر ده

```
npm start # Create React App لو مستخدم  
npm run dev # Vite لو مستخدم
```

هنا بقي هتلاقي المشروع شغال على `http://localhost:3000` أو `http://localhost:5173` لو Vite



فهم مبدأ ال Components في React

يعني إيه Components؟

هنا ال React مبنية على فكرة ال Components والتي بتخلي الكود متقسم لأجزاء صغيرة قابلة لإعادة الاستخدام بدل ما يكون كله في ملف واحد وبتقدر تتخيل ال Component زي قطعة Lego بتأخذها وتركبها في أي حته في التطبيق

أنواع ال Components في React

✓ **Function Components** (الأكثر استخداما حاليا).

✓ **Class Components** (كانت أساسية زمان بس دلوقتي ال Hooks خلت ال Function

Components هي الأساس)

هنا يجي دور ال Function Components (الطريقة الاحدث)

```
import React from "react";

const Welcome = () => {
  return <h1>صلى على النبي</h1>;
};

export default Welcome;
```

ده النوع الأبسط والأسهل وهو عبارة عن دالة بترجع ال JSX (كود شبه ال HTML)

- هنا عرفنا Component جديد اسمه Welcome بيرجع عنوان ال h1
- أي Component لازم يبدأ بحرف كابيتال (Welcome مش welcome)



ازاي استخدم ال Component جوا التطبيق

طبعا بعد ما عملنا ال Component بقي نقدر نستخدمه في أي مكان زي كده

```
import React from "react";
import Welcome from "./Welcome"; // استدعاء ال Component الي عملناه

const App = () => {
  return (
    <div>
      <Welcome />
    </div>
  );
};

export default App;
```

■ استدعينا ال Welcome بنفس طريقة ال HTML Tags

■ وأي Component بيتكتب كأنه وسم (</ Welcome>)

ال Class Components (مثل بنسخدمها كثير دلوقتي)

زمان React كانت بتعتمد على ال Class Components بس مع وجود ال Hooks الناس بقت تفضل ال Function Components بس لو صادفت كود قديم فده شكله

```
import React, { Component } from "react";

class Welcome extends Component {
  render() {
    return <h1>مثال مثلا؟</h1>;
  }
}

export default Welcome;
```

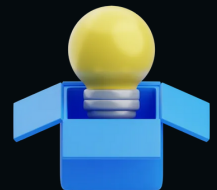
▲ الفرق هنا إننا استخدمنا class بدل function

▲ عندنا render () الي بيرجع JSX

▲ بنستخدم this جوه ال Class

خلاصة الكلام ؟

- ✓ ال Components بتقسم التطبيق لأجزاء صغيرة قابلة انك تعيد تستخدمها تاني
- ✓ عندنا Function Components (الأحدث والأسهل) و Class Components (قديم ومثل بنسخدمه كثير)
- ✓ تقدر تستخدم ال Component في أي مكان جوا التطبيق بنفس طريقة كتابة وسوم ال HTML



ال Props في React إزاي تبعت بيانات بين ال Components



يعني إيه Props؟

ال Props هي طريقة علشان تبعت بيانات من Component أب (Parent) إلى Component ابن (Child) علشان تقدر تتخيلها زي إنك بتبعت رسالة لشخص معين في نفس التطبيق بس لازم يقرأها بنفسه

طب ازاي نبعت Props ل Component؟

• هنا بنستخدم ال Props زي Attributes في ال HTML

```
import React from "react";
import Welcome from "./Welcome"; // استدعاء ال Component الابن

const App = () => {
  return (
    <div>
      <Welcome name="محمد" age={25} />
    </div>
  );
};

export default App;
```

▲ هنا استدعينا ال Welcome وبعتناله name و age ك Props

▲ name="محمد" عبارة عن نص (String) أما age={25} فهو رقم (Number)

إزاي نستخدم ال Props جوه ال Component؟

في ال Function Components ال Props بتكون عبارة عن كائن (Object) بييجي في ال Arguments بتاعة ال Function

```
import React from "react";

const Welcome = (props) => {
  return (
    <h1>
      سنة {props.age} عمرك {props.name}! أهلا يا
    </h1>
  );
};

export default Welcome;
```

▲ استخدمنا props.name و props.age علشان نظهر البيانات اللي جاية من ال Parent

▲ كل مرة تستدعي ال Component بقيم مختلفة هيتغير المحتوى بناءً على ال Props اللي تبعثت له

استخدام ال Destructuring لتسهيل الكود

هنا بدل ما نكتب **props.name** و **props.age** نقدر نستخدم ال **Destructuring** علشان نختصر الكود

```
const Welcome = ({ name, age }) => {
  return (
    <h1>
      سنة {age} عمرك {name}! أهلا يا
    </h1>
  );
};
```

ال Props في Class Components (مثش بنستخدمها كثير دلوقتي)

لو استخدمنا ال Class Components هحتاج نكتب **this.props** بدل **props** بس

```
import React, { Component } from "react";

class Welcome extends Component {
  render() {
    return <h1>يا أهلا {this.props.name}! عمرك {this.props.age} سنة </h1>;
  }
}

export default Welcome;
```

▲ طبعا نفس الفكرة بس بنستخدم **this.props** علشان نقدر نقرأ البيانات.

تحديد Default Props لو مفيش بيانات مبعوتة

لو ال **Component** ماخدش **Props** بنقدر نحدد قيم افتراضية (Default Props)

```
Welcome.defaultProps = {
  name: "ضيف",
  age: 20,
};
```

▲ لو استدعيت ال Component من غير ما تبعت name أو age، هيسخدم القيم الافتراضية اللي فوق

ال State في React
إزاي تخلي ال
Component
ديناميكي؟

يعني إيه State في React؟

ال State دي بتكون ذاكرة خاصة بأي Component بتسمح له يخزن بيانات تتغير مع الوقت بخلاف ال Props الي بتيجي من Parent Component ومش بتتغير. ال State ممكن تتغير جوه ال Component نفسه

إزاي نستخدم ال State؟

في React بنستخدم Hook اسمها **useState** عشان نقدر نضيف State لل Component

مثال:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>القيمة الحالية: {count}</p>
      <button onClick={() => setCount(count + 1)}>زياده</button>
    </div>
  );
}
```

هنا:

useState(0) بتحدد قيمة ابتدائية لل State وهي 0.

count القيمة الحالية.

setCount دالة لتغيير القيمة.

الحاجات المهمة الي تاخذ بالك منها؟

- ماينفعلش تغير ال State مباشرة، لازم تستخدم الدالة اللي بيقدمها **useState**
- أي تعديل في ال State بيخلي ال Component يتعمله إعادة Render

ملاحظات إضافية

- ممكن يكون ال State أي نوع من البيانات (...String, Number, Array, Object)
- لو بتتعامل مع Objects أو Arrays هنا استخدم Spread Operator (...) عشان تحافظ على البيانات القديمة

مثال مع Object:

```
const [user, setUser] = useState({ name: "Ali", age: 25 });  
  
setUser((prev) => ({ ...prev, age: 26 })); // تحديث السن مع انك تحفظ البيانات القديمة
```



التعامل مع الأحداث (Events) في (Handling) React

يعني إيه الأحداث (Events Handling) ؟

في React الأحداث (Events) شبيهة بالأحداث في HTML بس بدل **onclick** أو **onchange** بنكتبهم بـ camelCase زي **onClick** و **onChange**.

طب إزاي تتعامل مع الأحداث؟

هنا بنستخدم الأحداث مع أي عنصر JSX زي الأزرار أو الفورمات مثال:

```
function App() {  
  const handleClick = () => {  
    alert("إيه الدنيا يصحبي");  
  };  
  
  return <button onClick={handleClick}>اضغط هنا</button>;  
}
```

▲ هنا لما المستخدم يضغط على الزر هيتنفذ ال **handleClick** وهتطلع رسالة تنبيه

إزاي تبعت parameters للدوال

هنا بنستخدم الأحداث مع أي عنصر JSX زي الأزرار أو الفورمات مثال:

```
function App() {  
  const handleClick = (name) => {  
    alert(`${name} أهلا يا`);  
  };  
  
  return <button onClick={() => handleClick("أحمد")}>اضغط هنا</button>;  
}
```

▲ هنا بنستخدم **arrow function** عشان نبعت القيم

ازاي تمنع ال Prevent Default

مثلا لو عندك فورم وعاوز تمنع ال submit الافتراضي هتعمل كده

```
function App() {  
  const handleSubmit = (event) => {  
    event.preventDefault();  
    alert("!!الفورم مش هيتبعث");  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <button type="submit">إرسال</button>  
    </form>  
  );  
}
```

✓ أهم حاجات تركز عليها

1. الأحداث بتتكتب ب camelCase مش lowercase زي HTML
2. لو عاوز تبعت parameters استخدم **arrow function**
3. لو عاوز تمنع Prevent Default استخدم **event.preventDefault()**



The React logo, a stylized blue atom with three orbits and a central node, is positioned behind the text.

التعامل مع ال Conditional Rendering في React

يعني ايه Conditional Rendering ؟

في ال React ممكن تعرض مثلا Component أو عناصر معينة بناءً على شروط معينة وده اللي بنسميه Conditional Rendering

استخدام ال if العادي

ممكن نستخدم **if** جوه دالة عشان نحدد إذا كنا هنعرض حاجة ولا لأ.

```
function WelcomeMessage({ isLoggedIn }) {  
  if (isLoggedIn) {  
    return <h2>أهلا بيك يا مستخدم مسجل</h2>;  
  } else {  
    return <h2>!اتفضل سجل دخولك الأول</h2>;  
  }  
}
```

استخدام ال Ternary Operator

لو عاوز تخلي الكود أنظف شوية استخدم ال ternary operator (**?:**)

```
function WelcomeMessage({ isLoggedIn }) {  
  return isLoggedIn ? <h2>أهلا بيك</h2> : <h2>!سجل دخولك الأول</h2>;  
}
```

استخدام && (Short Circuiting)

مثال لو عاوز تعرض حاجة بس لما الشرط يكون **true** من غير **else**

```
function Notification({ hasMessages }) {  
  return (  
    <div>  
      <h2>!مرحبًا بك</h2>  
      {hasMessages && <p>عندك رسائل جديدة</p>}  
    </div>  
  );  
}
```

✓ النتيجة لو `hasMessages = true` هيظهر **"عندك رسائل جديدة"** غير كده مش هيظهر

استخدام switch في الحالات الكبيرة ؟

لو عندك أكثر من حالة ممكن تستخدم **switch**.

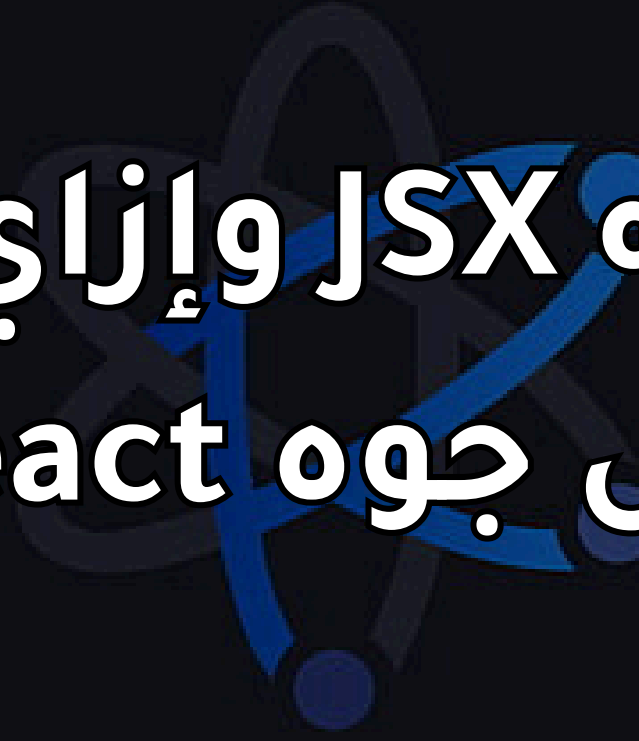
```
function StatusMessage({ status }) {  
  return <p>{{ success: "تم", error: "خطأ", loading: "جاري" }[status] || "غير معروف"}</p>;  
}
```

هنا:

if و **ternary operator** بيستخدموا للتحكم في عرض العناصر

&& مناسب لو عاوز تعرض حاجة بس لما الشرط يكون **true**

switch مفيد في الحالات اللي فيها أكثر من اختيار



يعني ايه JSX وإزاي
بيشتغل جوہ React؟

يعني ايه JSX؟

JSX هو اختصار لـ JavaScript XML وده عبارة عن طريقة تكتب بيها أكواد HTML جوه JavaScript وبيستخدمه React عشان يبقى الكود أسهل في الكتابة

طيب ليه نستخدم JSX؟

- بيخلي الكود أوضح وأقرب للـ HTML
- بيقلل الأخطاء لأن الـ JSX بيتحول لـ JavaScript حقيقي في الآخر
- بيشتغل بكفاءة مع الـ Virtual DOM في React

مثال عملي منغير JSX (كود عادي بالـ JavaScript)

```
const element = React.createElement('h1', null, 'Hello, React!');
ReactDOM.render(element, document.getElementById('root'));
```

باستخدام JSX (هنا الكود أنظف وأسهل)

```
const element = <h1>Hello, React!</h1>;
ReactDOM.render(element, document.getElementById('root'));
```

✓ لاحظ إننا بنكتب كود HTML جوه JavaScript وده بفضل JSX

حاجات مهمة تأخذ بالك منها عن JSX ؟

لازم يكون عندك **'import React from 'react'** لو بتستخدم **Class Components** لأن JSX في الآخر بيتحول لحاجة React بتفهمها ولو عندك أكثر من عنصر في JSX لازم تحطهم جوه عنصر واحد (زي <div> أو <Fragment>):

```
return (  
  <>  
    <h1>عنوان رئيسي</h1>  
    <p>فقرة توضيحية</p>  
  </>  
);
```

3 في ال JSX لازم تقفل كل التاجات حتى لو مفيهاش محتوى

✗ غلط → <"img src="image.png>

✓ صح → </ "img src="image.png>

✓ الخلاصة

1. JSX هو طريقة React لكتابة ال UI بكود شبه HTML جوه JavaScript

2. بيخلي الكود أوضح وأسهل من ال **React.createElement()**

3. بيتحول في الآخر لكود JavaScript عادي يفهمه المتصفح





ال Lifecycle Methods في React ؟

يعني إيه Lifecycle Methods؟

دي مجموعة دوال بتشتغل في مراحل مختلفة من حياة ال Component وبتساعدك تتحكم في ال Component وقت التحميل زي التحديث أو الحذف

مراحل ال Lifecycle؟

- مرحلة البداية (Mounting) ودي أول مرحلة لما ال Component بيظهر لأول مرة على الشاشة وأهم دالة فيها `componentDidMount()` → بتشتغل أول ما ال Component يتحمل ومفيدة لو هتجيب بيانات من API

```
class MyComponent extends React.Component {  
  componentDidMount() {  
    console.log("Component اتحمل!");  
  }  
  render() {  
    return <h1>؟ ازيك </h1>;  
  }  
}
```


التحديث والحذف؟

مرحلة التحديث (Updating)

دي بتحصل لما يحصل تغيير في ال State أو ال Props وأهم دوال فيها

componentDidUpdate(prevProps, prevState) → بتشتغل بعد كل تحديث لل Component

```
class MyComponent extends React.Component {  
  componentDidUpdate() {  
    console.log("Component اتحدث!");  
  }  
  render() {  
    return <h1>!مرحبا بك</h1>;  
  }  
}
```

مرحلة الحذف (Unmounting)

دي بتحصل لما ال Component يتم مسحه من الشاشة وأهم دالة فيها **componentWillUnmount()** → بتشتغل

قبل ما ال Component يتشال وبتستخدم لمسح ال Event Listeners أو تنصف أي بيانات مش محتاجها

```
class MyComponent extends React.Component {  
  componentWillUnmount() {  
    console.log("Component اتمسح!");  
  }  
  render() {  
    return <h1>!مرحبا بك</h1>;  
  }  
}
```

The React logo, a stylized blue atom with three orbits and three dots, is centered in the background.

التعامل مع ال Hooks في React

إيه هي ال Hooks؟

ال Hooks هي دوال بتسمح لك تستخدم State و Lifecycle Methods في Functional Components بدل ما تكون مضطر تستخدم Class Components

أشهر ال Hooks في React

1 useState() - عشان تخزن البيانات لل State

بتستخدم عشان تضيف State في Functional Components

2 useEffect() - بتنفذ الأكواد في أوقات معينة وبتشتغل لما يحصل تحديث في ال Component وبتشبه بظبط

ال componentDidMount و componentDidUpdate و componentWillUnmount في ال Class Components.

3 useContext() - بتشارك بيه البيانات بين المكونات وبتسمح لك تستخدم ال Context API بأسهل طريقة منغير ما

تحتاج تبعت ال Props لكل Component بنفسك

الخلاصة ✓

→ لتخزين البيانات **useState()**

→ بتشغل الاكواد لما يحصل تحديث. **useEffect()**

→ بتوصل منها للبيانات من ال Context API **useContext()**





التعامل مع ال Forms في React؟

يعني ايه ال Forms في React؟

ال Forms في React بتعتمد على ال State علشان تخزن البيانات اللي المستخدم بيكتبها أي تغيير في ال input بيتم تحديثه في ال State تلقائيًا

أهم الحاجات اللي لازم تعرفها:

- 1 ازاى تتحكم في القيم وكل input بيكون مربوط ب State وأي تعديل بيتم تحديثه لحظيًا
- 2 والتعامل مع الإرسال وهنا لازم تمنع تحديث الصفحة بال **e.preventDefault()**.
- 3 أكثر من input بدل ما تعمل useState لكل input ممكن تخزن كل البيانات في Object واحد

◆ مثال عملي

مثلا عندك input أي حاجة تكتبها بتظهر تحت منه بس لما تدوس إرسال البيانات بتظهر في console بدل ما الصفحة تتعمل reload.

SHIPPING ADDRESS

First Name *

Last Name *

Required

Required

Address Line 1 *

Required

Address Line 2

City

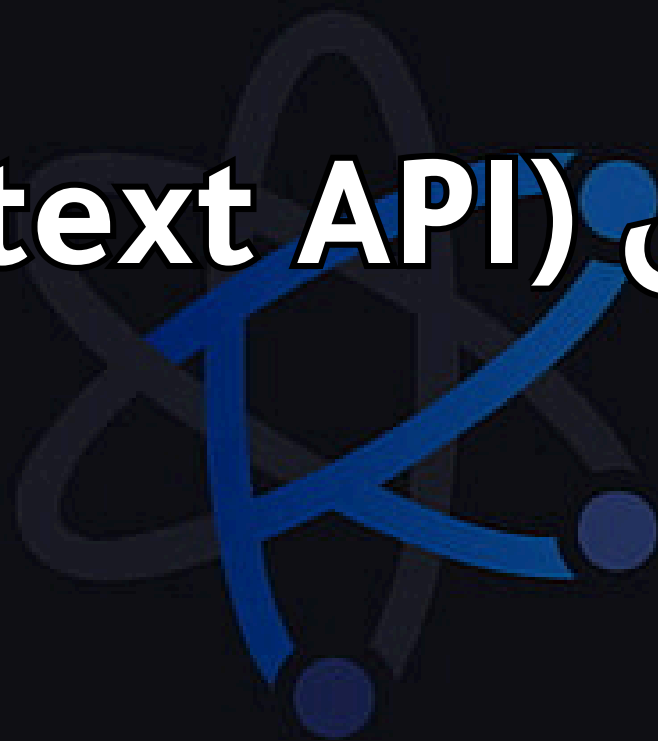
State

الخلاصة ✓

ان React بتخليك تتحكم في ال Forms بشكل أسهل وده بيساعدك في انك تحقق وتشوف البيانات قبل الإرسال



إدارة الـ Context API (Redux (Redux



يعني ايه ال (Context API أو Redux) ؟

لما المشروع يكبر التعامل مع البيانات بين المكونات يبقى صعب وموضوع ال props drilling مرهق علشان كده عندك Context API أو Redux علشان تسهل الموضوع

Context API ♦

- ده حل مدمج في React بيساعدك تبعت بيانات لأي مكون في التطبيق من غير ما تبعتها لكل مكون في السكة ومناسب لو عندك شوية بيانات زي ال theme أو ال user auth state وبتحتاج تستخدمها في أكثر من مكان ويشغل عن طريق ال Provider و Consumer بس مع ال Hooks بنستخدم useContext بدل ال Consumer القديم.

Redux ♦

- لو عندك بيانات بتتغير كثير أو التطبيق معقد هنا Redux بيكون حل عملي وبيعتمد على فكرة إن البيانات كلها بتكون في Store واحد وعلشان تغيرها بتستخدم Actions و Reducers وبيخليك تتابع كل تغيير حصل للبيانات (مفيد لو بتشتغل على تطبيق كبير زي e-commerce)

بس هنا هتسال نفسك أستخدم إيه؟ 

- لو التطبيق بسيط **استخدم Context API**.
- لو البيانات كثيرة وبتتغير بسرعة **استخدم Redux**.



The React logo, a stylized blue atom with three orbits and three dots, is positioned in the background of the slide.

التعامل مع REST APIs في React

يعني ايه REST APIs ؟

لما تعمل تطبيق React أكيد هتحتاج تجيب بيانات من API زي JSON Placeholder أو Backend خاص بيك هنا هتستخدم fetch أو مكتبات زي Axios عشان تتعامل مع الـ APIs

◆ استخدام fetch()

• JavaScript عندها fetch API المدمجة ودي طريقة سهلة تجلب البيانات من أي سيرفر

◆ استخدام Axios

• مكتبة قوية وأسهل من fetch لأنها بتدعم التعامل مع الأخطاء بشكل أفضل

• بتساعد في التعامل مع JSON تلقائياً بدل ما تستخدم json.() كل مرة

✓ شوية نصائح هتفيدك؟

- استخدم **useEffect()** عشان تجيب البيانات لما الكومبوننت يتحمل
- خزن البيانات في **useState()** عشان يتم إعادة تحديث الواجهة لما البيانات تتغير.
- التعامل مع الأخطاء (مثلاً لو السيرفر مش شغال أو النت قطع)



التنقل بين الصفحات في (React Router)

The logo for React Router is a stylized blue atom with three elliptical orbits and three dots representing electrons. It is positioned behind the text.

يعني ايه React Router ؟

لما تعمل تطبيق كبير هتحتاج تتنقل بين صفحات مختلفة منغير ما تحمل الصفحة من جديد وهنا بييجي دور على ال React Router تابع.....

✓ أول حاجة لازم تثبته

```
npm install react-router-dom
```

✓ بعد كده بتستخدمه بالشكل ده

```
import { BrowserRouter, Routes, Route, Link, useNavigate } from "react-router-dom";

const Home = () => <h2>الرئيسية</h2>;
const About = () => <h2>نحن</h2>;

const App = () => (
  <BrowserRouter>
    <nav><Link to="/"></Link> <Link to="/about"></Link></nav>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
    </Routes>
  </BrowserRouter>
);

export default App;
```

✓ اللي اتعلمناه هنا؟

- React Router بيخليك تتنقل بين الصفحات بدون ما تعمل Reload.
- <Link> أسرع وأفضل من <a>.
- <useNavigate()> بيساعدك تتنقل بالكود بسهولة.



أهم أسئلة ما بعد الملخص ؟

1. هل الملخص ده يغنييني عن الكورسات الكاملة؟

لا بيديلك الأساسيات اللي تساعدك تبدأ وتشتغل على مشاريع صغيرة وبعدها تقدر تتعمق في الكورسات احسن عشان التطبيق العملي

2. هل لازم أتعلم JavaScript الأول قبل React؟

أيوه 100%! React معتمدة بشكل أساسي على JavaScript فلو الأساسيات مش واضحة هتواجه عقبات قدام في التعلم

3. إيه الفرق بين React و Next.js؟

Next.js هو إطار مبني فوق React بس بيوفر ميزات إضافية زي ال SSR (Server-Side Rendering) وال SEO الأفضل

4. إمتى أستخدم Redux وإمتى أستخدم Context API؟

لو التطبيق صغير أو متوسط يبقى Context API كفاية بس لو التطبيق كبير ومعقد Redux بيكون حل أفضل لإدارة البيانات بشكل منظم

5. إزاي أطور مستوايا في React؟

- اشتغل على مشاريع عملي
- اقرأ ال Documentation الرسمي
- جرب مكتبات جديدة زي React Query و Zustand
- تابع الجديد في React

لو عجبك الملخص متنساش الدعاء ليك و لإخواتك
المسلمين خصوصا المستضعفين منهم و الاموات
منهم الي ملهمش حد يدعي لهم و متنساش طبعاً
كثرة الصلاة على النبي و جزاكم الله خيراً



اعداد: Youssef Salah